

Notes for ECE 223 Fall 13 Exam 2.

1: 19, 20, 2, 22, 15, 30, 11, 26, 9, 5

2.1: false

2.2: true

2.3: false

2.4: $(n+1)/2$ (integer division)

2.5: $2E/(n+1)$

2.6: $2^n - 1$

2.7: $\text{floor}(\log_2(n)) + 1$

2.8: 24, 16, 48, 88, 80, 64, 32

3.1: Level-order: 32, 20, 64, 16, 24, 48, 80, 88

3.2: MEX remains as root and the left subtree is unchanged. The level-order traversal of the right subtree is: SEA, ORD, TSP, MSP, SAN, ZRH

3.3: level-order: (H, L), A, J, P

3.4: level-order: 6, 4, 12, 2, 5, 8, 14

3.5: Following the left branch, the keys are: 5, 10, 15, 20, 40. There are no other changes

3.6: 10, 40, 16, 45, 43, 51, 21, 48, 49

4.1: $O(\log n)$

4.2: AVL tree. To form a list, each node stores $k+1$, where k is the number of nodes in the left subtree. Access is $O(1)$ but insertion and deletion are $O(n)$ for a sequential representation. For an AVL tree all operations are $O(\log n)$

4.3: make a BST by inserting these keys: 5, 3, 2, 4, 1. The search for the successor of any one of 1, 2, or

4. For example, if N points to the node with key 2, the value returned is null (since 2 does not have a right child). However its successor is 3.

5. Many solutions are possible. Here is one:

```
typedef struct params_t {
    int th_l, th_h;
    int small, med, large;
} params;

void dist_rec(bst_node_t *N, params *P) {
    if (N == NULL) return;
    if (N->key <= P->th_l) P->small++;
    else if (N->key >= P->th_h) P->large++;
    else P->med++;
    dist_rec(N->left, P);
    dist_rec(N->right, P);
}
```

to set up recursive call:

```
params results;
results.th_l = thresh_low;
results.th_h = thresh_high;
results.small = results.med = results.large = 0;
dist_rec(T->root, &results);
```